

תרגיל 1 - חלק רטוב

המתרגל האחראי על התרגיל: עדן הילה

שאלות על התרגיל - ב - Piazza בלבד.

הוראות הגשה:

- ההגשה בזוגות בלבד.
- עליכם לקרוא את נהלי תרגילי הבית תחת נהלי הקורס (הקובץ בעמוד "סילבוס" באתר הקורס)
- על כל יום איחור או חלק ממנו, שאינו באישור מראש, יורדו נקודות.
 - ניתן לאחר ב-3 ימים לכל היותר, ועל כל יום איחור יורדו 5 נקודות.
 - לבקשות מיוחדות להגשה באיחור יש לשלוח מייל למתרגל האחראי על התרגיל.
- הגשות באיחור רגיל (עם קנס) יתבצעו דרך אתר הקורס.
- יש להגיש את התרגיל בקובץ zip לפי ההוראות בעמוד הבא. **אי עמידה בהוראות אלו תעלה לכם בנקודות יקרות.**

הוראות והנחות לפתרון והגשת התרגיל

הוראות כלליות

- החומר הרלוונטי לתרגיל הוא עד וכולל שבוע 3 (החומר על קריאת פונקציות אינו נכלל ואסור לשימוש)
- אנא קראו את השאלות והתשובות בפיאצה - שכן הבהרות/מקרי קצה/הסברים נוספים שם נכללות כחלק חובה מהנחיות התרגיל.
- ישנם חמישה סעיפים - הפתרון של כל אחד מהסעיפים צריך להופיע בקובץ נפרד, ex1.asm, ex2.asm וכו' המצורפים לכם להשלמה.
- ההגשה שלכם צריכה להיות קובץ zip אחד שמכיל את חמשת הקבצים שקיבלתם, מבלי שהורדתם מהם כלום וכאשר הוספתם להם אך ורק פקודות אסמבלי ולא שום דבר אחר.

טסטים

- מסופק לכם קובץ בדיקה לתרגיל 4 והוראות כיצד להשתמש בו. הטסט מכסה מקרה בסיסי ומסופק לכם על מנת שתדעו כיצד אנו מצפים לקבל את הפתרונות (בכל אחד מחמשת התרגילים) וכיצד הם יבדקו.
- מומלץ לבצע טסטים נוספים בעצמכם לכל חמשת התרגילים, על בסיס מבנה טסט זה.

חוקים לכתיבת הקוד

- בקוד שאתם מגישים אין לשנות שום ערכים בזיכרון מלבד אלו שנאמר לכם מפורשות.
- עליכם לבצע כל חישוב אפשרי ברגיסטרים. ביצוע חישובים על ערכים בזיכרון אינו יעיל.
- בכל סעיף הניחו כי בתוויות הפלט מוקצה מספיק מקום ואל תניחו כלום על תוויות המקור שלא כתוב במפורש.
- על כל תווית שתשמשו בה לצורך המימוש להסתיים בארבעת התווים HW1_.

o לדוגמא -

```
loop: #this is a bad label name  
loop_HW1: #this is a good label name
```

זכרו: אי עמידה בכל אחת מההוראות הנ"ל עלול לגרור הורדת ניקוד לתרגיל וכמובן להשפיע על זכותכם לערעור במקרים מסוימים.

אופן בדיקת התרגיל וכתיבת טסטים בעצמכם

כל חמשת התרגילים יבדקו בצורה דומה. לכן, עקבו אחר ההוראות שיתוארו להלן, על מנת לבדוק את הקוד שלכם לפני ההגשה. הפורמט המתואר מחייב ואי עמידה בו יוביל להורדת נקודות (שימו לב - לא בהכרח נוכל להתחשב בכך בערעור).

בכל תרגיל, תקבלו קובץ asm המכיל section.text. בלבד. עליכם להשלים את הקוד שם. אין להוסיף sections נוספים לקובץ בעת ההגשה (ההגשה חייבת להכיל **section text** בלבד. בפרט **לא** להכיל את **section data**).

אז איך בכל זאת תוכלו לבדוק את התרגיל שלכם? זה פשוט. כאמור, לנוחותכם, צירפנו לתרגיל 4 טסט בודד בזיפ של קבצי הסטודנטים, היעזרו בו. הבדיקה היא בעזרת הקובץ run_test.sh שגם קיבלתם.

שימו לב שכל הטסטים על קבצי הקוד שלכם ירוצו עם timeout (כפי שגם מופיע ב-run_test.sh) ולכן כתבו אותם ביעילות. קוד שלא ייכתב ביעילות ולא יסיים את ריצתו על טסט מסוים עד ה-timeout, ייחשב כקוד שלא עמד בדרישות הטסט. בפרט הקוד ייבדק באותה מגבלת זמן שמופיעה ב-run_test.sh.

הערה:

לפני הרצת קבצי sh על המכונה, תצטרכו להריץ את הפקודה (פעם אחת בלבד):

```
chmod +x <your .sh file>
```

כדי לתת לקובץ הרשאות ריצה במערכת הפעלה.

הריצו את הקובץ run_test.sh:

```
./run_test.sh <path to asm file> <path to test file>
```

לדוגמה, עבור התרגיל 5 והטסט לדוגמה שלו, הריצו מתוך התיקייה שמכילה את קבצי הקוד של הסעיפים ואת תיקיית הטסטים את שורת הפקודה הבאה:

```
./run_test.sh ex5.asm ex5_example_test
```

כתיבת טסטים בעצמכם

לכל תרגיל תוכלו לכתוב טסט, שהמבנה שלו דומה למבנה של ex5_example_test, עם שינוי תוויות ובדיקות בהתאם.

תרגיל 1 (8 נק')

עליכם לממש את ex1 המוגדרת בקובץ ex1.asm.

בתרגיל זה תקבלו את ה- labels (תוויות) הבאים:

תווית מקור		תווית יעד	
שם	תוכן	שם	תוכן
Num1	int (4 בתיים)	BitCheck	בוליאני (בית אחד)
Num2	int (4 בתיים)		

עליכם לשים 1 ב-BitCheck אם לשני המספרים אותה **זוגיות (parity)** של מספר הביטים הדלוקים (מספר הביטים שערכם הוא '1').

- אם במספר אחד דולקים 6 ביטים, ובשני 8 ביטים: BitCheck = 1

- אם במספר אחד דולקים 13 ביטים ובשני 20 ביטים: BitCheck = 0

תרגיל 2 (12 נק')

עליכם לממש את ex2 המוגדרת בקובץ ex2.asm.

בתרגיל זה תקבלו את ה- labels (תוויות) הבאים:

תווית מקור		תווית יעד	
שם	תוכן	שם	תוכן
Address	כתובת זיכרון (8 בתים)	Result	בוליאני (בית אחד)
Length	unsigned int (4 בתים)		

אם המספר בגודל Length בתים החל מהכתובת השמורה ב-Address הוא פלינדרום של **בתים** (כלומר אם הופכים את סדר הבתים מהסוף להתחלה – הוא אותו מספר).

- מספר באורך 0 - הוא פלינדרום.

- המספר באורך 3: "0x570057" – הוא פלינדרום.

- המספר באורך 2: "0x3443" – אינו פלינדרום.

תוכלו להניח כי:

- $Length \geq 0$
- התוכן של Address הוא כתובת תקינה.
- בכתובת השמורה ב-Address יש **לפחות** Length בתים.

תזכורת: בית 1 = 8 ביטים = 2 ספרות בבסיס 16.

תרגיל 3 (25 נק')

עליכם לממש את ex3 המוגדרת בקובץ ex3.asm.
בתרגיל זה תקבלו את ה- labels (תוויות) הבאים:

תווית מקור		תווית יעד	
שם	תוכן	שם	תוכן
First	כתובת זיכרון (8 בתים) לאיבר הראשון ברשימה	Result	תוצאה (0/1/2) (בית אחד)

בתרגיל זה תקבלו רשימה מקושרת חד כיוונית דרך התווית **First**. איבר (node) ברשימה מכיל **לפי הסדר** הבא:

- מצביע לאיבר הבא (8 בתים)
- מידע של האיבר - int (4 בתים)

דוגמא לרשימה בת 2 איברים בתרגיל (ניתן לחזור על תרגול 3 עבור דוגמה נוספת לרשימה מקושרת):

```
first: .quad node1
node1: .quad node2
      .int 0x14
node2: .quad 0
      .int 0x15
```

האיבר האחרון ברשימה שומר במצביע לאיבר הבא שלו את הערך 0.
בתווית Result החזירו:

- '1' - אם איברי הרשימה הם סדרה חשבונית
- '2' - אם היא **אינה סדרה חשבונית אבל היא** סדרה חשבונית מדרגה שנייה
- '0' - אחרת

סדרה חשבונית מדרגה שנייה היא סדרה בה **ההפרשים** בין איבריה הם סדרה חשבונית.
לדוגמה – הסדרה 1,3,7,13,21 אינה סדרה חשבונית כי ההפרש לא קבוע, אבל ההפרשים בין איבריה – 2,4,6,8 הם סדרה חשבונית.

תרגיל 4 (25 נק')

עליכם לממש את ex4 המוגדרת בקובץ ex4.asm.

בתרגיל זה תקבלו את ה- labels (תוויות) הבאים:

תווית מקור		תווית יעד	
שם	תוכן	שם	תוכן
Lower	מחרוזת null-terminated	Upper	מחרוזת

עליכם לכתוב את המחרוזת השמורה ב-Lower ל-Upper ב-Capital Letters בלבד **וללא** תווים מיוחדים.

- המחרוזת מורכבת מאותיות באנגלית, מספרים, ותווים מיוחדים (%,# וכדומה).
- מספרים/אותיות Capital במחרוזת Lower מועתקים ללא שינוי.
- מוקצה מספיק מקום ב-Upper לכתיבה-שימו לב שאינכם צריכים להוסיף 'ט' בסוף המחרוזת.
- Upper ו-Lower הם מיקומים שונים בזיכרון

תרגיל 5 (30 נק')

עליכם לממש את ex5 המוגדרת בקובץ ex5.asm.

בתרגיל זה תקבלו את ה- labels (תוויות) הבאים:

תווית מקור		תווית יעד	
שם	תוכן	שם	תוכן
String	מחרוזת null-terminated	Result	תוצאה (3\2\1\0) (בית אחד)

עליכם להחזיר בתווית Result מספר לפי ההוראות הבאות:

- '1' - אם יש במחרוזת את אחת מהמילים * הבאות –
add, sub, mul, imul, div, idiv
- '2' - אם במחרוזת מופיעה מילה * באחת הצורות הבאות:
0x345 , 0345 , 345 , 0b345
 - 345 הוא רצף באורך כלשהו (לפחות 1) של ספרות כלשהן בלבד (התעלמו מהתאמת בסיס המספר לספרות – כלומר 0b ואחריו ספרות שאינן רק 1\0 תקין לצרכי התרגיל)
- '3' - אם שני התנאים מתקיימים.
- '0' - אחרת.

דוגמאות למחרוזות שיחזירו 1:

"mul 0xj6" , "n5 div %" , "aop 0bb4 add"

דוגמאות למחרוזות שיחזירו 2:

"0b123515 divn" , "%imul 0x23523623" , "ok 000 mul3"

דוגמאות למחרוזות שיחזירו 3:

, "mul 0x1234 whatsapp" , "hello idiv 18"

"mul mul 0x1 0b000 yo"

דוגמאות למחרוזות שיחזירו 0:

"dis multi" , "456xxx addrom" , "0xmul"

* מילה = מחרוזת שלפניה/אחריה רווחים או התחלת/סוף משפט. לדוגמה - add היא מילה במשפט "add a room" אבל היא לא מילה במשפט "address a room".

הערות אחרונות

איך בונים ומריצים לבד?

נתון לכם קובץ טסט לתרגיל 5 וקובץ run_test.sh. אתם יכולים לשנות את הקובץ הטסט של תרגיל 5 ולהשתמש במבנה שלו כדי לבדוק תרגילים אחרים באותה הצורה.

כדי להריץ, או לנפות שגיאות:

```
# run assembler (merge the 2 files into one asm before)
as ex5.asm ex5_example_test -o my_test.o
ld my_test.o -o ex5      #run linker
./ex5                   #run the code
gdb ex5                 #enter debugger with the code
```

את הקוד מומלץ לדבג באמצעות gdb. לא בטוחים עדיין איך? על השימוש ב-gdb תוכלו לקרוא עוד במדריך באתר הקורס. בהתחלה זה קשה, אבל זה יהיה שווה את זה! (ואם הסתבכתם - אנחנו זמינים בפיאצה).

- שימו לב - למכונה הווירטואלית של הקורס מצורפת תוכנת SASM, אשר תומכת בכתיבה ודיבוג של קוד אסמבלי וכן יכולה להוות כלי בדיקה בנוסף לגdb. (פגשתם אותה בתרגיל בית 0).
כתבו בcmd:

```
sasm <path_to_file>
```

כדי להשתמש ב-SASM לבנייה והרצת קבצי ה-asm, עליכם להחליף את שם התווית start_ בשם main (זאת מכיוון ש-sasm מזהה את תחילת הריצה על-ידי התווית main. **אל תשכחו להחזיר את start לפני ההגשה!**

ניתן לדבג באמצעות מנגנון הדיבוג של SASM במקום עם gdb, אך השימוש בו על אחריותכם **כלומר לא נתמך על ידנו ולא נתחשב בבעיות בעקבותו בערעורים**. שימו לב לשוני בין אופן ההרצה ב-SASM לאופן ההרצה שאנו משתמשים בו בבדיקה שלנו).

- בטסט אתם תפגשו את השורות הבאות

```
movq $60, %rax
movq $X, %rdi    # X is 0 or 1 in the real code
syscall
```

שורות אלו יבצעו exit(X) כאשר X הוא קוד החזרה מהתוכנית - 0 תקין ו-1 מצביע על שגיאה בקוד שאתם מגישים, אסור לפקודה syscall להופיע. קוד שיכיל פקודה זו, יקבל 0.